



SGCI

Frontend — Documentação Técnica Aprofundada

SPA em Angular 20 · Standalone + Signals · Análise do código-fonte

Angular 20.3

Standalone Components

Signals

RxJS 7.8

Gerado em 06/07/2026

Sumário

1. Panorama e princípios de design
2. Bootstrap e configuração global (`main.ts` , `app.config.ts`)
3. Estrutura de pastas
4. Camada `core` — configuração e modelos
5. Autenticação no cliente (`AuthService` + signals)
6. Interceptor de credenciais
7. Guards de rota (auth e admin)
8. Diretiva estrutural `*appRole`
9. Roteamento e lazy loading
10. Layout principal (sidebar, breadcrumbs, logout)
11. Features — visão geral e padrão comum
12. Feature Crises em profundidade (form + máquina de estados)
13. Dashboard, Relatórios e Auditoria
14. Fluxos de ponta a ponta

1. Panorama e princípios de design

O frontend do SGCI é uma **Single Page Application em Angular 20.3**, 100% standalone (sem `NgModule`), com estado reativo baseado em **signals** e um design system próprio em SCSS. Ele apenas renderiza as telas e consome a API REST do backend Quarkus.

Seis características que definem o front

#	Característica	Consequência prática
1	100% standalone — todo componente declara seus próprios <code>imports</code> .	Não há <code>AppModule</code> ; o bootstrap é direto no componente raiz.
2	Signals para estado (<code>signal</code> , <code>computed</code> , <code>effect</code>).	Reatividade fina sem <code>BehaviorSubject</code> ; a UI reage a login/logout automaticamente.
3	<code>inject()</code> em vez de injeção por construtor.	Código mais enxuto nos componentes/serviços.
4	Lazy loading em toda rota (<code>loadComponent</code>).	Cada tela vira um chunk carregado sob demanda.
5	Sessão por cookie (HttpOnly), não token no JS.	O cliente só espelha o usuário; a sessão real vive no servidor.
6	Sem biblioteca de UI (nada de Material/Bootstrap).	Design system manual em <code>styles.scss</code> , ícones SVG inline.

Stack

Tecnologia	Versão	Papel
Angular	20.3	Framework SPA (core, common, forms, router)
TypeScript	5.9	Linguagem
RxJS	7.8	Observables para HTTP e eventos de navegação
@angular/build	20.3	Application builder (novo)
SCSS	—	Design system próprio
Karma + Jasmine	—	Testes

2. Bootstrap e configuração global

A aplicação sobe sem módulos: `main.ts` monta o componente raiz `App` com um objeto de configuração. O `App` só contém um `<router-outlet>`.

```
// main.ts
bootstrapApplication(App, appConfig).catch((err) => console.error(err));
```

Todos os provedores globais ficam em `app.config.ts`:

```
// app.config.ts
export const appConfig: ApplicationConfig = {
  providers: [
    provideBrowserGlobalErrorListeners(),
    provideZoneChangeDetection({ eventCoalescing: true }),
    provideRouter(routes),
    provideHttpClient(withInterceptors([credentialsInterceptor])), // cookie em toda req.
  ],
};
```

Pontos importantes:

- **`provideRouter(routes)`** — registra as rotas lazy (seção 9).
- **`provideHttpClient(withInterceptors([ ... ]))`** — o `HttpClient` passa por um interceptor funcional que injeta as credenciais (o cookie de sessão) em toda requisição.
- **`provideZoneChangeDetection({ eventCoalescing:true })`** — detecção de mudanças baseada em Zone, agrupando eventos para reduzir ciclos.

A base da API está centralizada em `core/config/api.config.ts`: `export const API_BASE_URL = 'http://localhost:8080'`; — todos os serviços derivam suas URLs daí.

3. Estrutura de pastas

```
app/
├ app.ts / app.config.ts / app.routes.ts  (raiz)
├ core/                                   infraestrutura transversal
│   ├── config/      api.config.ts          (API_BASE_URL)
│   ├── guards/      auth.guard · admin.guard
│   ├── interceptors/ credentials.interceptor
│   ├── directives/  role.directive          (*appRole)
│   ├── models/      auth.model · user.model
│   └── services/    auth.service            (sessão via signals)
├ layouts/
│   └── main-layout/  casca das telas autenticadas
└ features/                                uma pasta por domínio
    ├── auth/         (login)
    ├── dashboard/
    ├── crises/       list · form · detail
    ├── relatorios/   list · form
    ├── auditoria/    list (admin)
    └ departamentos · tipos-crise · instituicoes · campi · cenarios
      (cada feature: models/ · pages/ · services/)
```

Cada feature isola `models` (interfaces/tipos), `pages` (componentes de tela) e `services` (acesso HTTP), mantendo alto o acoplamento interno e baixo entre features.

4. Camada `core` — configuração e modelos

O pacote `core` reúne tudo que é transversal: config da API, guards, interceptor, diretiva de papel, o serviço de autenticação e os modelos base.

MODELOS DE AUTENTICAÇÃO E USUÁRIO

```
// auth.model.ts
export interface LoginRequest { email: string; password: string; }
export interface LoginResponse { authenticated: boolean; message: string; }

// user.model.ts
export type Perfil = 'ADMIN' | 'COMUM';
export interface User { id: number; name: string; email: string; perfil: Perfil; }
```

Os tipos do front espelham exatamente os DTOs do backend — por exemplo, `Perfil` reflete o enum `PerfilUsuario`, e cada feature tem seus próprios tipos (ex.: `StatusCrise`, `NivelCriticidade`, `TipoAcao`) idênticos aos enums Java.

5. Autenticação no cliente — `AuthService`

O servidor guarda a sessão no cookie; o cliente apenas **espelha** o usuário atual em um signal. Todo o resto da aplicação lê esse signal de forma reativa.

```
@Injectable({ providedIn: 'root' })
export class AuthService {
  private readonly http = inject(HttpClient);
  private readonly baseUrl = `${API_BASE_URL}/auth`;

  readonly currentUser = signal<User | null>(null);
  readonly isAuthenticated = computed(() => this.currentUser() !== null);

  login(credentials: LoginRequest): Observable<LoginResponse> {
    return this.http.post<LoginResponse>(`${this.baseUrl}/login`, credentials);
  }
  logout(): Observable<void> {
    return this.http.post<void>(`${this.baseUrl}/logout`, {})
      .pipe(tap(() => this.currentUser.set(null))); // limpa o espelho local
  }
  fetchCurrentUser(): Observable<User> {
    return this.http.get<User>(`${this.baseUrl}/me`)
      .pipe(tap((user) => this.currentUser.set(user))); // atualiza o espelho
  }
}
```

Padrão-chave: `currentUser` é a única fonte de verdade da sessão no cliente. Os guards a atualizam via `fetchCurrentUser()`, a diretiva `*appRole` e o layout a leem, e o `logout()` a zera. Como é um signal, qualquer mudança propaga automaticamente para a UI.

LOGIN — O COMPONENTE

```
export class LoginComponent {
  readonly email = signal('');      readonly password = signal('');
  readonly errorMessage = signal<string | null>(null);
  readonly loading = signal(false);

  onSubmit(): void {
    this.loading.set(true);
    this.authService.login({ email: this.email(), password: this.password() }).subscribe({
      next: (r) => r.authenticated
        ? this.router.navigate(['/dashboard'])
        : this.errorMessage.set(r.message),
      error: () => this.errorMessage.set('Credenciais inválidas'),
    });
  }
}
```

6. Interceptor de credenciais

A API está em outra origem (8080) e a sessão é um cookie. Para o browser enviar esse cookie em requisições cross-origin, cada chamada precisa de `withCredentials: true`. Um interceptor funcional aplica isso a **todas** as requisições, de forma central:

```
export const credentialsInterceptor: HttpInterceptorFn = (req, next) => {
  return next(req.clone({ withCredentials: true }));
};
```

É o par natural da configuração de CORS do backend (`access-control-allow-credentials=true`): sem os dois lados alinhados, o cookie de sessão não trafegaria e toda requisição autenticada falharia com 401.

7. Guards de rota

Os guards são funções (`CanActivateFn`) e, em vez de confiar num flag local, **revalidam a sessão no backend a cada navegação** chamando `GET /auth/me`.

AUTHGUARD — EXIGE SESSÃO VÁLIDA

```
export const authGuard: CanActivateFn = () => {
  const authService = inject(AuthService);
  const router = inject(Router);
  return authService.fetchCurrentUser().pipe(
    map(() => true), // sessão ok → libera
    catchError(() => of(router.parseUrl('/login'))), // erro → redireciona
  );
};
```

ADMINGUARD — EXIGE PERFIL ADMIN

```
export const adminGuard: CanActivateFn = () => {
  ...
  return authService.fetchCurrentUser().pipe(
    map((user) => user.perfil === 'ADMIN' ? true : router.parseUrl('/dashboard')),
    catchError(() => of(router.parseUrl('/login'))),
  );
};
```

Assim há três desfechos: autenticado e autorizado → segue; autenticado sem permissão → volta ao painel; não autenticado → login. Como efeito colateral útil, `fetchCurrentUser()` também popula o signal `currentUser`, deixando a UI pronta assim que a rota carrega.

8. Diretiva estrutural `*appRole`

Controla a **visibilidade visual** de elementos conforme o perfil, reagindo a login/logout sem código extra — porque observa o signal `currentUser` dentro de um `effect()`.

```
@Directive({ selector: '[appRole]', standalone: true })
export class RoleDirective {
  readonly appRole = input.required<string[] | string>();
  private rendered = false;

  constructor() {
    effect(() => {
      const roles = this.appRole();
      const allowed = Array.isArray(roles) ? roles : [roles];
      const user = this.auth.currentUser();
      this.toggle(!user && allowed.includes(user.perfil)); // mostra/esconde
    });
  }
  // toggle: cria ou limpa a embedded view via ViewContainerRef
}
```

Uso no template: `*appRole="[ 'ADMIN' ]"`. Importante: isso é apenas *cosmético* — esconde botões e itens de menu. A proteção real vem dos guards (front) e das anotações `@RolesAllowed` (back).

9. Roteamento e lazy loading

Todas as telas usam `loadComponent` (carregamento sob demanda). As rotas autenticadas são *filhas* do `MainLayoutComponent`, que aplica `authGuard` uma vez e vale para todas.

```
export const routes: Routes = [
  { path: '', redirectTo: 'login', pathMatch: 'full' },
  { path: 'login', loadComponent: () => import('.../login.component') ... },
  {
    path: '',
    loadComponent: () => import('.../main-layout.component') ... ,
    canActivate: [authGuard], // protege todos os filhos
    children: [
      { path: 'dashboard', loadComponent: ... },
      { path: 'crises', loadComponent: ... },
      { path: 'crises/nova', loadComponent: ... }, // antes de :id
      { path: 'crises/:id', loadComponent: ... },
      { path: 'relatorios', loadComponent: ... },
      { path: 'relatorios/novo', loadComponent: ... },
      { path: 'auditoria', canActivate: [adminGuard], loadComponent: ... }, // só ADMIN
      ... // departamentos, campi, cenarios, etc.
    ],
  },
  { path: '**', redirectTo: 'login' },
];
```

Rota	Componente	Guard
/login	LoginComponent	—
/dashboard	DashboardComponent	authGuard
/crises · /crises/nova · /crises/:id	List · Form · Detail	authGuard
/relatorios · /relatorios/novo	List · Form	authGuard
/departamentos · /tipos-crise · /instituicoes · /campi · /cenarios	*ListComponent	authGuard
/auditoria	AuditoriaListComponent	+ adminGuard

Ordem importa: crises/nova é declarada antes de crises/:id , senão "nova" seria capturada como um id.

10. Layout principal (MainLayoutComponent)

É a casca de todas as telas autenticadas: sidebar de navegação, cabeçalho com breadcrumbs, cartão do usuário e botão de logout. O `<router-outlet>` renderiza a página atual no centro.

BREADCRUMBS REATIVOS

A trilha é reconstruída a cada `NavigationEnd`, traduzindo os segmentos da URL por um mapa de rótulos:

```
const CRUMB_LABELS = {
  dashboard: 'Painel', crises: 'Crises', nova: 'Nova Crise',
  relatorios: 'Relatórios', auditoria: 'Auditoria', ...
};
constructor() {
  this.updateCrumbs(this.router.url);
  this.router.events.pipe(
    filter(e): e is NavigationEnd => e instanceof NavigationEnd),
    takeUntilDestroyed(),
  ).subscribe(e => this.updateCrumbs(e.urlAfterRedirects));
}
// segmentos numéricos (ex.: /crises/5) viram "Detalhe"
```

VISIBILIDADE POR PAPEL NO MENU

O menu usa `*appRole=["ADMIN"]` para esconder itens de não-admins. Assim, um usuário COMUM vê só o grupo *Principal* (sem "Nova Crise"), enquanto o ADMIN vê também todo o bloco *Administração*.

Grupo	Itens	Visível para
Principal	Painel, Crises, Relatórios	Todos autenticados
Principal	Nova Crise	ADMIN
Administração	Departamentos, Tipos de Crise, Cenários, Campi, Instituições, Auditoria	ADMIN

```
// logout: chama o backend e volta ao login
onLogout(): void {
  this.authService.logout().subscribe(() => this.router.navigate(['/login']));
}
```

11. Features — visão geral e padrão comum

Cada feature segue a mesma anatomia: um `service` (`providedIn: 'root'`, usa `inject(HttpClient)` e deriva a URL de `API_BASE_URL`) e componentes de página com signals `loading` / `error` / dados carregados no `ngOnInit`.

SERVIÇO TÍPICO (CRISES)

```

@Injectable({ providedIn: 'root' })
export class CriseService {
  private readonly baseUrl = `${API_BASE_URL}/api/crises`;
  listAll() { return this.http.get<Crise[]>(this.baseUrl, ...); }
  findById(id) { return this.http.get<Crise>(`... /${id}`, ...); }
  create(req) { return this.http.post<Crise>(this.baseUrl, req, ...); }
  changeStatus(id,s) { return this.http.patch(`... /${id}/status`, { status: s }, ...); }
  listAcoes(id) { return this.http.get(`... /${id}/acoes`, ...); }
  createAcao(id,req) { return this.http.post(`... /${id}/acoes`, req, ...); }
}

```

PANORAMA DAS FEATURES

Feature	Páginas	Endpoints consumidos
auth	login	/auth/login
dashboard	dashboard	/api/dashboard/resumo
crises	list · form · detail	/api/crises (+ status, acoes)
relatorios	list · form	/api/relatorios
auditoria	list (admin)	/api/auditoria (+ facets)
departamentos · tipos-crise · instituicoes · campi · cenarios	list	/api/{ref}

Observação de estado atual: os serviços dos 5 cadastros de referência já expõem CRUD completo (`create` / `update` / `deactivate`), mas os componentes de lista atualmente só chamam `listAll()` — funcionam como listagens somente-leitura. Apenas **crises** e **relatórios** têm telas de formulário.

12. Feature Crises em profundidade

12.1 Formulário de nova crise

É a tela mais rica: carrega em paralelo as quatro fontes dos selects e monta o `CriseRequest`. O `nivel` começa em `BAIXO` e as FKs começam nulas.

```
export class CriseFormComponent implements OnInit {
  readonly tipos = signal<TipoCrise[]>([]);  readonly campi = signal<Campus[]>([]);
  readonly cenarios = signal<Cenario[]>([]);  readonly departamentos = signal<Departamento[]>([]);
  readonly niveis: NivelCriticidade[] = ['BAIXO', 'MEDIO', 'ALTO', 'CRITICO'];

  model: CriseRequest = { titulo:'', descricao:'', nivel:'BAIXO',
    tipoCriseId:null, campusId:null, cenarioId:null, departamentoId:null };

  ngOnInit() {                                     // popula os 4 dropdowns
    this.tipoCriseService.listAll().subscribe((d) => this.tipos.set(d));
    this.campusService.listAll().subscribe((d) => this.campi.set(d));
    this.cenarioService.listAll().subscribe((d) => this.cenarios.set(d));
    this.departamentoService.listAll().subscribe((d) => this.departamentos.set(d));
  }
  onSubmit() {
    this.criseService.create(this.model).subscribe({
      next: () => this.router.navigate(['/crises']),
      error: () => this.error.set('Não foi possível cadastrar a crise ... '),
    });
  }
}
```

12.2 Detalhe da crise — máquina de estados no cliente

A tela de detalhe implementa o ciclo de vida da crise usando um mapa `fluxo` e dois `computed`: um para o próximo status, outro para decidir se o formulário de nova ação aparece.

```

private readonly fluxo: Record<StatusCrise, StatusCrise | null> = {
  ABERTA: 'EM_ANDAMENTO', EM_ANDAMENTO: 'RESOLVIDA',
  RESOLVIDA: 'ENCERRADA', ENCERRADA: null,           // terminal
};
readonly proximoStatus = computed(() => {
  const atual = this.crise();
  return atual ? this.fluxo[atual.status] : null;
});
readonly formVisivel = computed(() => {           // só registra ação se aberta/andamento
  const s = this.crise()?.status;
  return s === 'ABERTA' || s === 'EM_ANDAMENTO';
});

avancarStatus() {
  const proximo = this.proximoStatus();
  if (!proximo) return;
  this.criseService.changeStatus(this.criseId, proximo)
    .subscribe({ next: (d) => this.crise.set(d), ... });
}
registrarAcao() {
  this.criseService.createAcao(this.criseId, this.novaAcao)
    .subscribe({ next: () => { this.novaAcao = {descricao:'',tipo:'CONTENCAO'}; this.carregarAcoes(); }, ... });
}

```

Essa máquina de estados é o *espelho* da regra do backend (crise ENCERRADA é terminal). O front esconde o botão de avançar/registrar; o back a garante de fato — defesa em profundidade.

13. Dashboard, Relatórios e Auditoria

DASHBOARD

Página única (landing pós-login e destino do `adminGuard` para não-admins). Consome `GET /api/dashboard/resumo` e exibe KPIs (`totalCrises` , `crisesAbertas` , `crisesEmAndamento` , `crisesResolvidas` , `crisesCriticas`) que linkam para as crises.

RELATÓRIOS

No formulário, o select de crises é **filtrado** para exibir apenas crises `RESOLVIDA` / `ENCERRADA` (`crisesElegiveis`), espelhando a regra do backend de que só crises fechadas geram relatório.

AUDITORIA (SÓ ADMIN)

Lista com filtros por entidade, ação e usuário — os valores dos dropdowns vêm de `GET /api/auditoria/facets` . Usa `DatePipe` para a data/hora e helpers que colorem badges por ação (CRIAR→verde, ATUALIZAR→azul, EXCLUIR→vermelho) e por faixa de status HTTP.

14. Fluxos de ponta a ponta

Autenticação e proteção de rota

- 1 Usuário envia login → AuthService.login() → POST /auth/login
- 2 Backend responde com Set-Cookie: jwt (HttpOnly) → navega para /dashboard
- 3 A cada navegação guardada:
authGuard → fetchCurrentUser() → GET /auth/me (cookie enviado pelo interceptor)
└─ sucesso → popula currentUser + libera
└─ erro → redireciona a /login
- 4 adminGuard (só /auditoria): exige perfil === 'ADMIN', senão volta a /dashboard
- 5 UI reativa: *appRole e o layout leem o signal currentUser
- 6 Logout → POST /auth/logout → currentUser.set(null) → /login

Operação sobre uma crise

- 1 ADMIN em /crises/nova preenche o form (4 selects carregados em paralelo)
→ CriseService.create() → POST /api/crises → volta a /crises
- 2 Qualquer autenticado abre /crises/:id (detalhe + ações)
- 3 Registrar ação (form visível só se ABERTA/EM_ANDAMENTO)
→ createAcao() → POST /api/crises/{id}/acoes
- 4 ADMIN avança status (computed proximoStatus)
→ changeStatus() → PATCH /api/crises/{id}/status
- 5 Após RESOLVIDA/ENCERRADA: /relatorios/novo lista só crises elegíveis